

Seksjon 1

Om eksamen

DAT108 Programmering og Webapplikasjoner, høst 2023

Tid: 4 timer

Du kan svare på engelsk eller norsk. Skriver du på norsk kan du bruke engelske faguttrykk.

Jeg har lest og forstått instruksjonene

☐ Ja

☐ Nei

Oppgave 1 (12%) - Strømmer/Streams

Om oppgave A, B, C, D, E og F:

I hver deloppgave skal du lage en strøm (stream) som løser et problem gitt i oppgaveteksten.

Kort om retting:

- Teller ca. 12% (30 minutter).
- Hver deloppgave teller ca. 2% (4-5 minutter)
- Oppgaven rettes automatisk.
- Det er bare en **helt** rett løsning på hvert problem.
- Du blir gitt **delvis** uttelling for delvis korrekt svar:
 - Alle felt som er korrekte gir poeng.
 - Feil svar gir **IKKE** trekk.

Eksempel: Gitt en liste av ord (`List<String> ordInn`), lag en liste som inneholder kun de ordene som begynner med bokstaven "A".

```
 begynnerMedA = ordInn. ()  
. ()  
. ( );
```

Dette er en øvingsoppgave. Den gir ikke poeng og du kan sjekke svaret ditt så mange ganger du vil.

Check Answer

Oppgave A: Gitt en liste av ord (`List<String> ordInn`), lag en ny liste (`ordUt`) som kun inneholder ord som har odd lengde.

```
 ordUt = ordInn. ()  
. ()  
. ( );
```

Oppgave B: Gitt en liste med gaver (List<Gave> onskeliste), finn ut om alle de 10 billigste gavene er myke.

```

    erMyke = onskeliste.  

        .  

    )  

    .  

    )  

    .  

    )  

    .  

    );

```

Om klassen Gave:

Klassen Gave har følgende feltvariablene (med tilsvarende get'ere og set'ere):

- String navn; // Navnet på gaven (For eksempel "Byggeklusser", "Genser" osv...)
- Integer pris; // Pris (i NOK)
- Integer vekt; // Vekt (i gram)
- Boolean myk; // Sann om gaven er myk, usann om gaven er hard

Oppgave C: Gitt en liste med gaver (List<Gave> onskeliste), blandt de 20 billigste gavene, finn en hard gave blandt de 10 tyngste gavene (dersom en slik gave eksisterer).

```

    hardGave = onskeliste.  

        .  

    )  

    .  

    )  

    .  

    )  

    .  

    )  

    .  

    )  

    .  

    )  

    .  

    );

```

Oppgave D: Gitt en liste av ønskelister (List<List<Gave>> innOnskelister), skriv ut navnene på alle de myke gavene fra de ønskelistene som har færre enn 10 gaver.

```

    innOnskelister.  

        .  

        .  

        .  

        .  

        .
    );

```

Oppgave E: Gitt en liste med ønskelister (List<List<Gaver>> ønskelisteliste), finn snittprisen på gavene i den lengste ønskelisten. Hvis du får en tom liste av ønskelister eller hvis den lengste ønskelisten er tom så skal du returnere -1.

```

    snittpris = ønskelisteliste. ( )
    . (
    )
    .orElse ( new ArrayList<Gave> ( )
    );
    . ( )
    . (
    )
    . (
    )
    . ( )
    .orElse ( -1 );

```

Oppgave F: Lag et akronym/en forkortelse fra en liste med strenger (se eksempelet under).

```

    akronym = strenger. ( )
    . (
    );

```

Eksempel:

- ["United", "States of", "America"] blir til "U.S.A."
- ["American", "Standard", "Code for", "Information", "Interchange"] blir til "A.S.C.I.I."
- ["light", "amplification by", "", "stimulated", "", "emission of", "radiation"] blir til "l.a.s.e.r." (Tomme strenger i listen fjernes).

Antagelser/bemerkninger:

Dette feltet kan stå tomt. Dersom noe er uklart i oppgavene over så kan du skrive ned eventuelle antagelsene du har gjort her. Marker kommentarene dine med oppgavenummer. Skriv for eksempel:

// Oppgave B iii).

Dine antagelser...

B *I* U | ≡ ≡ á

0 / 10000 Word Limit

Oppgave 2 (8%) - Funksjonelle kontrakter

Om oppgave A, B, C og D:

I hver oppgave skal du bruke nedtrekksmenyen til å lage **gyldige** lambdauttrykk med **rett type** som **løser** problemet som oppgaveteksten beskriver.

Kort om retting:

- Teller ca. 8% (20 minutter).
- Hver deloppgave teller ca. 2% (4-5 minutter)
- Oppgaven rettes automatisk.
- Det er bare en **helt** rett løsning på hvert problem.
- Du blir gitt **delvis** uttelling for delvis korrekt svar:
 - Alle felt som er korrekte gir poeng.
 - Feil svar gir **IKKE** trekk.

Eksempel:

// i) Hva er typen til lambdauttrykket lagret som variabelen eksempel1?

eksempel1 = x -> x.startsWith("a");

// ii) Lag et lambdauttrykk som skriver ut lengden til en gitt streng. Hva er typen til dette lambdauttrykket?

eksempel2 = ;

Dette er to øvingsoppgaver. De gir ikke poeng og du kan sjekke svaret ditt så mange ganger du vil.

Check Answer

Oppgave A: Gi rett type til variablene a1, a2 og a3 under:

// i)

a1 = (x, y) -> x + x*y + y;

// ii)

a2 = String::length;

// iii)

a3 = x -> x.toUpperCase().length();

Oppgave B: Velg det lambdauttrykket passer med typen til variabelene b1, b2 og b3.

// i)

Predicate<String> b1 = ;

// ii)

Function<Integer, String> b2 = ;

// iii)

Comparator<String> b3 = ;

Oppgave C: Hvilket funksjonelle grensesnitt har lambdauttrykkene c1, c2, c3, c4 og c5, gitt at...

```
// i)
// Når vi skriver c1.apply("Kari") så returneres et objekt av typen Person.
```

c1;

```
// ii)
// Når vi skriver c2.apply("Kari", 20) så returneres et objekt av typen Person.
```

c2;

```
// iii)
// Når vi skriver c3.test(new Person("Kari", 20)) så returneres verdien true.
```

c3;

Oppgave D: Velg rett lambdauttrykk og type til variablene d1, d2 og d3.

```
// i)
// Inn: En person (Person).
// Ut: Etternavnet til personen.
```

d1 = ;

```
// ii)
// Inn: To personer (Person).
// Ut: Aldersforskjellen (differansen) mellom personene (Hint: Metoden skal brukes i
en strøm til sortering av en liste av Personer).
```

d2 = ;

```
// iii)
// Inn: En person (Person) og et lambdauttrykk som regner ut inntektsskatt for en pe
rson.
// Ut: true dersom personen betaler mindre enn 500 000 NOK i skatt, ellers false.
```

d3 =
 ;

Antagelser/bemerkninger:

Dette feltet kan stå tomt. Dersom noe er uklart i oppgavene over så kan du skrive ned eventuelle antagelsene du har gjort her. Marker kommentarene dine med oppgavenummer. Skriv for eksempel:

// Oppgave B iii).

Dine antagelser...

B	<i>I</i>	<u>U</u>		☰	¹ / ₂ ☰	á
						0 / 10000 Word Limit

Oppgave 3 (15%) - Tråder og trådsikkerhet

Om oppgaven:

Denne oppgaven består av fire deloppgaver, A, B, C og D. Oppgaven rettes manuelt og hver av deloppgavene teller omtrent like mye.

Koden under skulle simulere tre ulike typer nisser som arbeider *parallellelt* på et ju leverksted:

- Lagenisse, som lager leker (2 stk.)
- Pakkenisse, som pakker inn leker (3 stk.)
- Julenisse, som leverer pakker (1 stk.)

PROBLEMET ER AT KODEN UNDER IKKE OPPFØRER SEG SOM VI HADDE TENKT.

Målet med denne oppgaven er at du skal finne ut av hva det er som har gått galt og at du skal foreslå en løsning.

Ønsket oppførsel

Krav til programmet:

- Alle lekene skal ha et unikt nummer.
- Alle pakkene skal ha et unikt nummer.
- Alle lekene skal pakkes inn nøyaktig en gang.
- Alle pakkene skal leveres nøyaktig en gang.
- Leker skal ikke pakkes inn før de er lagd.
- Pakker skal ikke leveres før de er pakket inn.
- Når det er tomt for leker skal pakkenissen vente på neste leke.
- Når det er tomt for pakker skal julenissen vente på neste pakke.
- Når det ikke er plass til flere leker skal lagenissen vente med å legge fra seg 1 eken til en pakkenisse har fjernet en leke.
- Når det ikke er plass til flere pakker skal pakkenissen vente med å legge fra seg pakken til julenissen har fjernet en pakke.
- Utover dette skal nissene **ikke** vente på hverandre.

Vi tillater at:

- Lekene ikke blir skrevet ut i sortert rekkefølge.
- Pakkene ikke blir skrevet ut i sortert rekkefølge.
- En leke har samme nummer som en pakke.
- En pakke inneholder en leke med et annet nummer enn det pakken har.

Eksempel på godkjent utskrift:

Leke 1
Leke 2
Leke 4
Leke 3
Pakke 1 [Leke 2]
Pakke 2 [Leke 1] osv...

Eksempel på feil i utskrift:

Leke 1
Leke 2
Leke 3
Leke 3
Pakke 1 [Leke 2]
Pakke 1 [Leke 1] osv...

(Det er nøyaktig to feil her: 1) at "Leke 3" er skrevet ut to ganger og 2) at "Pakke 1" er skrevet to ganger)

KODE

Klassen Juleverksted:

```
1 | import java.util.concurrent.*;
2 |
3 | class Juleverksted {
4 |     private final BlockingQueue<String> leker;
5 |     private final BlockingQueue<String> pakker;
6 |     private int lekeTeller = 0;
7 |     private int pakkeTeller = 0;
8 |
9 |     Juleverksted(int kapasitet) {
10 |         leker = new ArrayBlockingQueue<>(kapasitet);
11 |         pakker = new ArrayBlockingQueue<>(kapasitet);
12 |     }
13 |
14 |     // Metode som simulerer at en nisse lager en leke.
15 |     public void lagLeke() throws InterruptedException {
16 |         Thread.sleep(2000); // Simu
lerer arbeid
17 |         ++lekeTeller;
18 |         String leke = "Leke " + lekeTeller;
19 |         System.out.println(leke);
20 |         leker.put(leke);
21 |     }
22 |
23 |     // Metode som simulerer at en nisse pakker inn en leke.
24 |     public void pakkInnLeke() throws InterruptedException {
25 |         String leke = leker.take(); // Simu
lerer arbeid
26 |         Thread.sleep(3000);
27 |         ++pakkeTeller;
28 |         String pakke = "Pakke " + pakkeTeller + " [" + leke + "]";
29 |         System.out.println(pakke);
30 |         pakker.put(pakke);
31 |     }
32 |
33 |     // Metode som simulerer at en nisse leverer en pakke.
34 |     public void leverPakke() throws InterruptedException {
35 |         String pakke = pakker.take(); // Simu
lerer arbeid
36 |         Thread.sleep(1000);
37 |         System.out.println("Leverer : " + pakke);
38 |     }
39 | }
40 |
```

Klassen Lagenisse:

```
41 | class Lagenisse implements Runnable {
42 |     private final Juleverksted juleverksted;
43 |
44 |     Lagenisse(Juleverksted juleverksted) {
45 |         this.juleverksted = juleverksted;
46 |     }

```

```

47 |
48 |     @Override
49 |     public void run() {
50 |         while (true) {
51 |             try {
52 |                 juleverksted.lagLeke();
53 |             } catch (InterruptedException e) {
54 |
55 |             }
56 |         }
57 |     }
58 | }
59 |

```

Klassen Pakkenisse:

```

60 | class Pakkenisse implements Runnable {
61 |     private final Juleverksted juleverksted;
62 |
63 |     Pakkenisse(Juleverksted juleverksted) {
64 |         this.juleverksted = juleverksted;
65 |     }
66 |
67 |     @Override
68 |     public void run() {
69 |         while (true) {
70 |             try {
71 |                 juleverksted.pakkInnLeke();
72 |             } catch (InterruptedException e) {
73 |
74 |             }
75 |         }
76 |     }
77 | }
78 |

```

Klassen Julenisse:

```

79 | class Julenisse implements Runnable {
80 |     private final Juleverksted juleverksted;
81 |
82 |     Julenisse(Juleverksted juleverksted) {
83 |         this.juleverksted = juleverksted;
84 |     }
85 |
86 |     @Override
87 |     public void run() {
88 |         while (true) {
89 |             try {
90 |                 juleverksted.leverPakke();
91 |             } catch (InterruptedException e) {
92 |
93 |             }
94 |         }

```

```
95 |     }
96 | }
97 |
```

Klassen JuleverkstedEksempel (main metode):

```
98 | // Main metoden
99 | public class JuleverkstedEksempel {
100 |     public static void main(String[] args) {
101 |         Juleverksted juleverksted = new Juleverksted(10); // Maksimal kapasit
102 |         et på verkstedet
103 |
104 |         // Oppretter 1 Julenisse, 2 Lagenisser og 3 Pakkenisser
105 |         Thread julenissen = new Thread(new Julenisse(juleverksted));
106 |         Thread lagenisse1 = new Thread(new Lagenisse(juleverksted));
107 |         Thread lagenisse2 = new Thread(new Lagenisse(juleverksted));
108 |         Thread pakkenisse1 = new Thread(new Pakkenisse(juleverksted));
109 |         Thread pakkenisse2 = new Thread(new Pakkenisse(juleverksted));
110 |         Thread pakkenisse3 = new Thread(new Pakkenisse(juleverksted));
111 |
112 |         julenissen.start();
113 |         lagenisse1.start();
114 |         lagenisse2.start();
115 |         pakkenisse1.start();
116 |         pakkenisse2.start();
117 |         pakkenisse3.start();
118 |     }
119 | }
```

Oppgave A:

Pek på de linjene i koden hvor du finner kappløpstilstander (race conditions) som du mener er problematiske. Forklar hva det er som kan gå galt.

B *I* U | ≡ ≡ á

0 / 500 Word Limit

Oppgave B:

Hvorfor er det en dårlig ide å fikse problemene vi har ved å bruke synkronisering på metodene i koden over?

B *I* U | ≡ ≡ á

0 / 500 Word Limit

Oppgave C:

Hvordan kunne vi gjort koden over trådsikker? Velg deg en metodene over som ikke er trådsikker og gjør den trådsikker (uten å over-synkronisere).

B <i>I</i> <u>U</u> ☰ ☷ á
0 / 500 Word Limit

Oppgave D:

På hvilke kodelinjer ville vi fått problemer dersom vi hadde brukt en Queue istedenfor en BlockingQueue? Forklar hva det er som kan gå galt.

B <i>I</i> <u>U</u> ☰ ☷ á
0 / 500 Word Limit

Oppgave 4 (10%) - JavaScript teorispørsmål

I disse flervalgsoppgavene kan ingen, ett eller flere av utsagnene være korrekte. En oppgave gir ingen poeng hvis et av de feile utsagnene er valgt, selv om også et riktig valg er gjort. Helgardering gir kun poeng hvis alle utsagnene er korrekte.

a) (2 %)

En webside skal kjøre JavaScript-koden under:

```
window.alert('Velkommen');
```

Hvilke av alternativene under kan brukes for å få utført-koden?

- ☐ Lagre koden i en fil *index.js*. En webside *index.html* vil alltid laste JavaScript-kode fra en fil *index.js*.

Lagre koden i en fil *index.js*. Legg så inn følgende HTML i HTML-koden:

- ☐ `<script src="index.js"></script>`

Legg inn i HTML-koden:

- ☐ `<code>window.alert('Velkommen')</code>`

Legg inn i HTML-koden:

- ☐ `<script>window.alert('Velkommen')</script>`

Legg inn i HTML-koden:

- ☐ `<script javascript="window.alert('Velkommen')"></script>`

Lagre koden i en fil *index.js* og legg inn i HTML-koden:

- ☐ `<script>index.js</script>`

b) (2 %)

Hvilke JavaScript-objekt kan gi oss operativsystemet til klienten?

- ☐ window
- ☐ navigator
- ☐ history
- ☐ screen
- ☐ document

c) (1 %)

En webside skal kjøre JavaScript-koden under:

```
const liListe = document.getElementsByTagName("li");
while (liListe.length > 0) {
  liListe[0].remove();
}
```

Hvilke av utsagnene under er korrekte?

- ☐ Koden forsøker å fjerne samme HTML *LI* element hver gang.
- ☐ Koden vil feile da argumentet *"li"* til *getElementsByTagName* ikke er en lovlig CSS-selektor.
- ☐ Nettleser vil oppdatere hvilket element i listen som er første element og lengden av listen fortløpende, og løkken vil stoppe når siste element er fjernet.
- ☐ Løkken vil aldri stoppe da *liListe.length* ikke blir oppdatert.
- ☐ Koden vil feile siden tagg-navn må skrives med store bokstaver, dvs. *LI* for tagg **.
- ☐ JavaScript kan legge til HTML elementer, men ikke fjerne element. Koden vil derfor feile.

d) (1 %)

En webside skal kjøre JavaScript-koden under:

```
const liListe = document.querySelectorAll("li");
while (liListe.length > 0) {
  liListe[0].remove();
}
```

Hvilke av utsagnene under er korrekte?

- ☐ Koden forsøker å fjerne samme HTML *LI* element hver gang.
- ☐ Koden vil feile da argumentet *"li"* til *getElementsByTagName* ikke er en lovlig CSS-selektor.
- ☐ Nettleser vil oppdatere hvilket element i listen som er første element og lengden av listen fortløpende, og løkken vil stoppe når siste element er fjernet.
- ☐ Løkken vil aldri stoppe da *liListe.length* ikke blir oppdatert.
- ☐ Koden vil feile siden tagg-navn må skrives med store bokstaver, dvs. *LI* for tagg **.
- ☐ JavaScript kan legge til HTML elementer, men ikke fjerne element. Koden vil derfor feile.

e) (2 %)

Hva gjør koden under?

```
let [a,b] = data;
```

- ☐ Koden er ulovlig da vi ikke kan bruke JSON syntaks for Array på venstre side av et likhetstegn.
- ☐ Koden returnerer **true** hvis begge sider av likhetstegnet er en **Array** med likt innhold.
- ☐ Koden tilordner variabel *a* til verdien til *data*, mens *b* blir *undefined*.
- ☐ Variabel *data* må være en liste-struktur med nøyaktig to elementer.
- ☐ Variabel *data* må være en liste-struktur med minst to elementer.
- ☐ Koden vil tilordne verdier til variablene *a* og *b* fra elementer i *data* som må være en liste-struktur.

f) (2 %)

Koden under er gitt:

```
summer(...[1,4,2,8]);
```

Hvilke av utsagnene under er korrekte?

- ☐ Konstruksjonen med tre punktum «...» er ulovlig.
- ☐ Antall punktum angir hvor mange elementer som skal hentes ut fra listen med tall. Funksjonen *summer* kjøres derfor med argumentene 1, 4 og 2.
- ☐ Funksjonen *summer* kjøres med 4 argumenter der de tre første er tegnet punktum «.», og siste argument er en liste av tall.
- ☐ Funksjonen *summer* kjøres med 4 argumenter som er 1, 4, 2, og 8.
- ☐ Funksjonen *summer* kjøres med 4 argumenter der de tre første er **undefined** og siste er en liste av tall.
- ☐ Funksjonen *summer* kjøres med 4 argumenter der de tre første er **null** og siste er en liste med tall.

Antagelser/bemerkninger:

Dette feltet kan stå tomt. Dersom noe er uklart i oppgavene over så kan du skrive ned eventuelle antagelsene du har gjort her. Marker kommentarene dine med oppgavenummer. Skriv for eksempel:

// Oppgave b).

Dine antagelser...

B *I* u | ☰ ☷ á

0 / 10000 Word Limit

Oppgave 5 (15%) - JavaScript program

Program for å registrerer unike heltall.

Se oppgave på papir. For å svare velger du «Administrer vedlegg». Så

1. Nytt vedlegg
2. Kode
3. Kodespråk - JavaScript

Kode

I oppgaven skal dere fylle inn den manglende koden i en JavaScript-klasse **Talldata**. Se oppgavetekst på papir for detaljer.

```
class Talldata {  
    #tallinput  
  
    // Legg til her eventuelle flere private felt  
  
    constructor(root) {  
        this.#tallinput =  
            root.getElementsByTagName("input")[0];  
        const bt =  
            root.getElementsByTagName("button")[0];  
  
        bt.addEventListener("click",  
            (event) => { this.#lestall(event) }  
        );  
  
        // Legg inn her mer kode hvis nødvendig  
    }  
  
    #lestall(event) {  
        // Legg inn kode her  
    }  
  
    // Legg til eventuelle flere private metoder  
}  
  
const rootelement = document.getElementById("root");  
new Talldata(rootelement);
```

B *I* U | ≡ ≡ ≡ á

10000 Word Limit

Oppgave 6 (40%) - Web backend med Spring MVC

Oppgaven består av 4 delspørsmål a)..d). Se selve oppgaveteksten på papir.

For å svare, velger du "Administrer vedlegg". Så Nytt vedlegg | Kode | Kodespråk Java (evt. noe annet på a)

Svar helst på alle delspørsmål i samme vedlegg.

</>